



Wykład 1: Wiersz poleceń i filozofia Uniksa

dr inż. Bartosz Czech

Katedra Genetyki · Uniwersytet Przyrodniczy we Wrocławiu

`bartosz.czech@upwr.edu.pl`

Plan i cel

Tryb **blended** — większość nauki online, we własnym tempie, na Moodle.

Element	Opis
2 wykłady wprowadzające	Na żywo, na starcie — ogólne wprowadzenie do całości (to jeden z nich)
10 modułów online	Samodzielna praca na Moodle: materiały i zadania
Zaliczenie pisemne	Na żywo, na ostatnim spotkaniu

Wykłady wprowadzające dają obraz całości i wspólny język. Właściwa praca odbywa się w modułach — to w nich ćwiczą Państwo konkretne narzędzia.

Zakres materiału

- 1 Powłoka, potoki i filtry tekstowe
- 2 Przetwarzanie tekstu: `sed` i `awk`
- 3 Skrypty bash — podstawy
- 4 Skrypty bash — zagadnienia zaawansowane
- 5 Bazy, formaty i interwały genomowe
- 6 Reprodukowalność: Git, Conda, Docker
- 7 Python i Biopython
- 8 Python w praktyce: własne narzędzia
- 9 Snakemake: budowa pipeline'ów
- 10 Analiza i wizualizacja w R oraz Quarto

Fundament to warsztat bash (moduły 1–4), dalej dane (5), reprodukowalność (6), Python (7–8), automatyzacja (9) i wizualizacja (10).

Cały kurs na uczelnianej platformie **Moodle** — poza dwoma wykładami na żywo, jedyne potrzebne miejsce.

- **Logowanie** — tym samym kontem co poczta uczelniana; kurs pojawia się po zapisaniu na przedmiot
- **Materiały** każdego modułu: wprowadzenie, przykłady i dane do zadań
- **Zadania** — treści i miejsce oddania rozwiązań
- **Ogłoszenia** — komunikaty prowadzącego i terminy oddania

Reguła praktyczna

Warto włączyć powiadomienia z kursu i zaglądać na Moodle regularnie — tam obowiązują terminy oddania zadań.

- **Moduł 0** — przygotowanie środowiska: instrukcje instalacji dla każdego systemu
- **Moduły 1–10** — właściwy kurs: materiały, dane, treści zadań i miejsce oddania

Slajdy obu wykładów wprowadzających (PDF) są dostępne na stronie bczech.github.io/teaching — nie trzeba niczego przepisywać w trakcie.

Kurs jest samodzielny, ale wymaga **systematyczności** — najlepiej pracować regularnie przez cały semestr.

- Kolejne moduły są budowane na poprzednich, więc warto przerabiać je po kolei
- Na jeden moduł warto zarezerwować **kilka godzin** pracy własnej rozłożonych na tydzień
- Rozwiązania oddaje się na bieżąco, żeby nie kumulować zaległości

Reguła praktyczna

Najczęstszą przyczyną kłopotów nie jest trudność materiału, lecz odkładanie modułów na koniec. Fundament (moduły 1–4) trzeba przerobić na bieżąco, bo bez niego nie ruszą kolejne części.

1. **Materiały** — czytają Państwo wprowadzenie z przykładami
2. **Zadania** — wykonują Państwo listę zadań albo mini-projekt na dostarczonych danych
3. **Oddanie** — przesyłają Państwo rozwiązanie na Moodle w terminie

Oddawany plik

Zwykle jeden plik z użytymi poleceniami/skryptami i istotnymi fragmentami wyników, nazwany według wzoru **M#_nazwisko_album** (np. **M3_kowalski_123456.txt**).

Każdy moduł kończy się namacalnym wynikiem. Suma tych wyników to znacząca część oceny końcowej.

Ocena końcowa

- 40% — zadania z 10 modułów
- 60% — zaliczenie pisemne

Próg zaliczenia: 50%

Skala ocen

Wynik	Ocena
$\geq 90\%$	5,0
80–89%	4,5
70–79%	4,0
60–69%	3,5
50–59%	3,0
$< 50\%$	2,0

Zadania oddaje się w trakcie kursu; zaliczenie pisemne jest na końcu.

- Termin: **do ustalenia** ze studentami
- Zakres: **całość kursu**, od wiersza poleceń po pipeline'y i wizualizację
- Forma: **pytania koncepcyjne** oraz **czytanie i pisanie** poleceń i fragmentów skryptów
- Języki: bash, **awk** i Python

Tryb pracy

- Kurs **samodzielny** — bez formalnej listy obecności na modułach
- Warunkiem zaliczenia są **oddane zadania** i **zaliczenie pisemne** w wyznaczonych terminach

Konsultacje: poniedziałek 14:00–15:30, pokój 306 w Katedrze Genetyki
(proszę o wcześniejszy kontakt mailowy: bartosz.czech@upwr.edu.pl)

Pomoc: forum przedmiotu na Moodle oraz konsultacje. Pytania o materiał i zadania warto zadawać na bieżąco, a nie tuż przed terminem.

Podręczniki

- [Buffalo](#) — *Bioinformatics Data Skills* (O'Reilly)
- *Linux. Programowanie w powłoce* (rozdziały o `sed`/`awk`)

Darmowe, dostępne online

- Software Carpentry — *The Unix Shell*
- Kross — *The Unix Workbench*
- Janssens — *Data Science at the Command Line*
- MIT — *The Missing Semester*
- [explainshell.com](#) — rozkłada polecenie na części

Plan dzisiejszego wykładu

- 1 Po co bioinformatykowi wiersz poleceń
- 2 Instalacja i konfiguracja środowiska
- 3 Terminal, powłoka i budowa polecenia
- 4 Praca z plikami i katalogami
- 5 Filozofia Uniksa i potoki
- 6 Dane i formaty w bioinformatyce
- 7 Dobre praktyki i typowe pułapki

1. Po co bioinformatykowi wiersz poleceń

Pracownia Informatyczna — praktyczny warsztat codziennej pracy bioinformatyka z danymi.

- Nie uczymy programowania dla samego programowania — uczymy **warsztatu**: jak sprawnie i powtarzalnie przetwarzać dane biologiczne
- Punktem wyjścia jest **wiersz poleceń**, a docelowo także skrypty, kontrola wersji, środowiska, kontenery i pipeline'y
- Kurs jest **samodzielny** — zaczynamy od podstaw terminala i budujemy warstwa po warstwie

Cel praktyczny

Po tym kursie potrafią Państwo wziąć zbiór plików z danymi i przetworzyć je własnym, powtarzalnym rozwiązaniem, zamiast klikać ręcznie po jednym pliku.

- Pojedynczy eksperyment sekwencjonowania to **miliony** odczytów i pliki rzędu **gigabajtów**
- Plik FASTQ z jednej próbki RNA-seq ma zwykle kilka–kilkanaście GB; projekt to dziesiątki takich plików
- Genom człowieka w postaci tekstowej to około **3 miliardy** znaków

Dlaczego nie arkusz kalkulacyjny

Excel zatrzymuje się na ok. milionie wierszy i otwiera cały plik w pamięci. Danych bioinformatycznych zwykle nie da się w nim ani otworzyć, ani przetworzyć.

Interfejs graficzny (GUI)

- Praca przez klikanie w okna i ikony
- Intuicyjny na start
- Każda czynność wykonywana ręcznie, raz
- Trudno opisać i powtórzyć krok po kroku

Wiersz poleceń (CLI)

- Praca przez wpisywanie poleceń tekstowych
- Wymaga nauki, ale daje pełną kontrolę
- Polecenie można zapisać i uruchomić ponownie
- Łatwo automatyzować i skalować

CLI = *Command Line Interface*, czyli tekstowy interfejs wiersza poleceń.

- **Reprodukowalność** — zapisane polecenie jest dokładnym opisem tego, co zrobiono; da się je powtórzyć i sprawdzić
- **Automatyzacja** — tę samą operację można wykonać na tysiącach plików jednym poleceniem
- **Skala** — narzędzia czytają dane strumieniowo, więc rozmiar pliku nie ogranicza pracy
- **Praca zdalna** — serwery i klastry obliczeniowe udostępniają zwykle wyłącznie terminal

Większość profesjonalnych narzędzi bioinformatycznych w ogóle nie ma interfejsu graficznego — działają tylko z wiersza poleceń.

Praca przez klikanie

„Otworzyłem plik, zazaczyłem kolumnę, wybrałem z menu sortowanie, potem odfiltrowałem...”

Po miesiącu nikt — łącznie z autorem — nie odtworzy dokładnie tych kroków.

Praca przez polecenia

Jedna linia, którą można wkleić do publikacji, wysłać koledze i uruchomić za rok z tym samym wynikiem.

```
sort -k3 -n dane.tsv > posortowane.tsv
```

- Poważne obliczenia uruchamia się na **zdalnych serwerach** i **klastrach**, nie na własnym laptopie
- Łączymy się z nimi przez **SSH** — dostajemy sam wiersz poleceń, bez okien i myszy
- Pliki przesyłamy poleceniami (**scp**, **rsync**); zadania zlecamy systemowi kolejkowemu

Klaster i system kolejkowy SLURM — moduł dziewiąty.

Excel — nie tylko za mały; bywa wręcz **niebezpieczny** dla danych biologicznych.

- Nazwy genów zamieniane są na daty: **SEPT2** → „2 września”, **MARCH1** → data
- Identyfikatory tracą zera wiodące, długie liczby zaokrąglane są do notacji wykładniczej
- Zmiany są **ciche** — plik zapisuje się bez ostrzeżenia, a błąd wychodzi dużo później

Udokumentowany problem

Badanie z 2016 roku wykazało, że około **jedna piąta** publikacji genetycznych z załącznikami w Excelu zawierała tak przekłamane nazwy genów. Skala problemu skłoniła później do zmiany części symboli genów.

Typowa analiza — **ciąg kroków**; wynik jednego narzędzia jest wejściem następnego.

1. Kontrola jakości surowych odczytów (**FastQC**)
2. Przycięcie odczytów słabej jakości (**fastp, cutadapt**)
3. Dopasowanie do genomu odniesienia (**bwa, STAR**)
4. Zliczenie i wykrycie różnic (**samtools, bcftools**)
5. Analiza statystyczna i wykresy (**R**)

Dlaczego to ważne dla kursu

Każdy z tych kroków uruchamia się z wiersza poleceń, a całość spina się w jeden powtarzalny **pipeline**.

Właśnie takie łączenie narzędzi ćwiczymy przez cały semestr.

Sztandarowe programy bioinformatyczne — **wyłącznie** z wiersza poleceń.

Narzędzie	Zastosowanie
bwa, bowtie2, STAR	dopasowanie odczytów do genomu
samtools, bcftools	obróbka dopasowań i wariantów
bedtools	operacje na interwałach genomowych
salmon, kallisto	kwantyfikacja ekspresji
seqkit, fastp	obróbka plików FASTA/FASTQ

Nauka wiersza poleceń to warunek wstępny korzystania z tych programów — nie ma dla nich „wersji z oknami”.

2. Instalacja i konfiguracja środowiska

Na starcie — **jedno spójne środowisko** dla wszystkich, niezależnie od systemu na własnym komputerze.

- Instrukcje instalacji znajdą Państwo w **Module 0** na Moodle — dziś omawiamy je z lotu ptaka
- Konfigurację robi się **raz**, na początku semestru; potem po prostu działa
- Jeśli coś nie zadziała, jest czas i miejsce na pomoc — o tym na końcu tej części

Cel

Po tej konfiguracji każdy z Państwa ma działający terminal uniksowy, menedżer pakietów Conda i edytor kodu — ten sam zestaw, na którym oparte są wszystkie moduły.

Element	Do czego
Terminal uniksowy	Miejsce, w którym wpisujemy polecenia (bash)
Conda (Miniforge)	Instaluje narzędzia i izoluje środowisko kursu
Edytor (VS Code)	Pisanie i podgląd skryptów oraz plików tekstowych
Git	Kontrola wersji — przyda się od Modułu 6

Terminal to *gdzie* pracujemy, Conda to *skąd* bierzemy narzędzia, a edytor to *czym* piszemy skrypty. Te trzy rzeczy wystarczą na cały kurs.

Który system operacyjny?

Punkt startu zależy od systemu; cel wspólny — działający **bash** i **conda**.

System	Co robimy
Linux	Terminal jest wbudowany — od razu instalujemy Conde
macOS	Terminal wbudowany + narzędzia wiersza poleceń, potem Conda
Windows	Instalujemy WSL (podsystem Linuksa), w nim pracujemy jak na Linuksie

Windows: nie PowerShell, nie CMD

Na Windows **nie pracujemy** w PowerShellu ani wierszu poleceń CMD. Polecenia kursu są uniksowe, więc potrzebujemy prawdziwego Linuksa przez WSL.

WSL (Windows Subsystem for Linux) — pełny Linux wewnątrz Windowsa, bez maszyny wirtualnej i osobnego rozruchu.

- Dostają Państwo prawdziwy terminal z **bash** i menedżerem pakietów Ubuntu
- Pliki Windows i Linux są wzajemnie dostępne, więc można korzystać z obu światów
- To **rekomendowany** sposób pracy na Windows — ten sam, którego używa się zawodowo

Po instalacji WSL wszystko, co pokazujemy w tym i kolejnych wykładach, działa u Państwa identycznie jak na Linuksie i macOS.

Menu Start → **PowerShell jako administrator**, jedno polecenie:

```
wsl --install
```

1. Polecenie instaluje WSL i domyślnie dystrybucję **Ubuntu**
2. Po instalacji **restart** komputera
3. Przy pierwszym uruchomieniu Ubuntu zakładają Państwo **nazwę użytkownika i hasło** (osobne od konta Windows)
4. Od tej pory terminal Ubuntu otwierają Państwo z menu Start

To jedyny moment, w którym używamy PowerShella — wyłącznie po to, by zainstalować WSL. Dalej pracujemy już w terminalu Ubuntu.

macOS wywodzi się z Uniksa — terminal **wbudowany** (aplikacja *Terminal* albo *iTerm2*).

- Podstawowe narzędzia deweloperskie instaluje jedno polecenie:

```
xcode-select --install
```

Uwaga na starą wersję bash

Domyślny **bash** w macOS ma wersję 3.2 (z 2006 r.) i nie obsługuje części nowszej składni. Dlatego nowszy **bash** zainstalujemy przez Conda i to w nim będziemy pracować.

Terminal i menedżer pakietów — część systemu; zostaje tylko doinstalować Condę.

- Terminal otwierają Państwo skrótami (często **Ctrl+Alt+T**)
- Menedżer pakietów zależy od dystrybucji: **apt** (Ubuntu/Debian), **dnf** (Fedora)
- Narzędzia systemowe aktualizuje się poleceniem menedżera, np. **sudo apt update**

Użytkownicy Linuksa mają najkrótszą drogę: przechodzą od razu do instalacji Condy na następnym slajdzie.

Conda — instaluje narzędzia bioinformatyczne i trzyma je w izolowanych **środowiskach**.

- Instaluje pakiety **bez uprawnień administratora** i bez ręcznego kompilowania
- Każdy projekt może mieć własne środowisko z własnymi wersjami narzędzi
- Dzięki temu **u wszystkich** działa dokładnie ta sama wersja — to filar reprodukowalności

Miniforge

Instalujemy wariant **Miniforge** — lekki, darmowy i domyślnie korzystający z otwartego kanału **conda-forge**. Do niego wracamy szerzej w Module 6.

Instalator dla swojego systemu (link w Module 0), uruchamiany w terminalu:

```
bash Miniforge3-$(uname)-$(uname -m).sh
```

1. Instalator prowadzi krok po kroku (akceptacja licencji, lokalizacja)
2. Na koniec zgadzają się Państwo na inicjalizację powłoki
3. **Zamykają i otwierają terminal ponownie**, żeby zmiany zadziałały
4. Po ponownym otwarciu na początku znaku zachęty pojawia się **(base)**

Napis **(base)** oznacza, że Conda działa i jesteśmy w jej środowisku bazowym.

Osobne środowisko dla kursu, zamiast środowiska bazowego:

```
conda create -n pracownia python git bash
conda activate pracownia
```

- `conda create -n pracownia ...` — tworzy środowisko o nazwie **pracownia** z wybranymi narzędziami
- `conda activate pracownia` — wchodzi do niego; znak zachęty pokazuje wtedy **(pracownia)**
- Dokładną listę pakietów i gotowe polecenie znajdą Państwo w Module 0

Reguła praktyczna

Przed pracą nad modulem warto sprawdzić, czy w znaku zachęty widać **(pracownia)**. Jeśli nie — `conda activate pracownia`.

- **Visual Studio Code** — darmowy, standard w pracy zawodowej
- Składnia, podgląd plików i wbudowany terminal w jednym oknie
- Rozszerzenie **WSL** (Windows) — praca wprost na plikach w Linuksie
- Rozszerzenia **Python** i **R** — druga połowa kursu

Sprawdzenie, czy narzędzia odpowiadają — w aktywnym środowisku **pracownia**:

```
bash --version      # powinno pokazać wersję 5.x
conda --version     # np. conda 24.x
git --version       # np. git 2.4x
python --version    # np. Python 3.12
```

- Każde polecenie powinno wypisać **numer wersji**, a nie komunikat o błędzie
- **command not found** oznacza, że dany element nie jest jeszcze zainstalowany albo aktywny

Objaw	Zwykła przyczyna
<code>wsł</code> nie działa	wyłączona wirtualizacja w BIOS lub brak aktualizacji Windows
<code>conda: command not found</code>	nie otwarto terminala ponownie po instalacji
brak <code>(base)</code> w wierszu	powłoka nie została zainicjalizowana
<code>bash</code> wciąż 3.2 (macOS)	pracujemy poza środowiskiem <code>conda</code>

Większość problemów rozwiązuje ponowne otwarcie terminala i sprawdzenie, czy środowisko jest aktywne.

Problemy z konfiguracją są normalne — ważne, żeby rozwiązać je **zanim** ruszą moduły.

- **Moduł 0 na Moodle** — instrukcje krok po kroku ze zrzutami ekranu dla każdego systemu
- **Forum przedmiotu** — proszę opisać system, polecenie i pełną treść błędu
- **Konsultacje** — jeśli utknie problem, przyjdą Państwo z laptopem i rozwiążemy na miejscu

Reguła praktyczna

Konfigurację środowiska warto zrobić w pierwszym tygodniu. Nerozwiązany problem instalacyjny na dwa dni przed terminem Modułu 1 to niepotrzebny stres.

3. Terminal i powłoka

- **Unix** powstał na przełomie lat 60. i 70. w Bell Labs jako system dla programistów
- Zbudowano go wokół prostej idei: małe narzędzia, wspólny format tekstowy, łączenie w potoki
- **Linux** (lata 90.) to darmowa, otwarta implementacja tych samych zasad; dziś napędza serwery, klastry i większość infrastruktury nauki
- macOS również wywodzi się z Uniksa, więc jego terminal działa niemal identycznie

Dlaczego to nadal aktualne

Zasady sprzed pół wieku okazały się trwałe: te same polecenia, których uczymy się dziś, działają na laptopie, na serwerze i na superkomputerze. Nauka opłaca się na całą karierę.

Terminal

Okno, w którym wpisujemy tekst i czytamy odpowiedzi. Sam w sobie nic nie liczy.

Powłoka (shell)

Program, który czyta nasze polecenia i je wykonuje. Najczęściej **bash**.

Polecenie

Konkretna instrukcja, np. „wypisz zawartość katalogu” albo „policz wiersze”.

Znak zachęty (*prompt*), zwykle \$, oznacza, że powłoka czeka na polecenie. Wpisujemy je i zatwierdzamy Enterem.

Powłoka to zwykły program — jest ich kilka; różnią się drobiazgami składni, nie ideą.

Powłoka	Uwagi
sh	pierwotna, minimalna; wzorzec zgodności
bash	najpopularniejsza na serwerach i w kursach; tej używamy
zsh	domyślna w macOS, wygodniejsza w codziennej pracy

macOS

Systemowy **bash** w macOS został przy wersji 3.2 z 2006 roku i nie zna części nowszej składni. Dlatego w kursie pracujemy w **bash** z Condy, jednakowym u wszystkich.

System	Sposób
Linux	aplikacja „Terminal” albo skrót Ctrl+Alt+T
macOS	Terminal (w katalogu Narzędzia) albo przez Spotlight
Windows	WSL — podsystem Linuksa; instalujemy go w Module 0

Windows

Nie korzystamy z PowerShell ani z „wiersza polecenia” **cmd** — mają inne polecenia. WSL daje prawdziwe środowisko Linux wewnątrz Windows, identyczne z tym na serwerach.

Kilka nieszkodliwych poleceń na rozgrzewkę — po prostu coś wypiszą:

```
whoami      # nazwa zalogowanego użytkownika
date        # bieżąca data i godzina
echo "Witaj" # wypisz podany tekst
clear       # wyczyść ekran terminala
```

- Żadne z nich nic nie psuje — służą do oswojenia się z wpisywaniem poleceń
- `echo` przyda się później do wypisywania wartości i budowania komunikatów w skryptach
- `clear` (albo `Ctrl+L`) porządkuje ekran, gdy zrobi się na nim tłoczno

Sam prompt niesie już informacje, zanim wpisujemy polecenie:

```
bartek@laptop:~/dane/fasta$
```

Fragment	Znaczenie
bartek	nazwa użytkownika
laptop	nazwa komputera (własny czy zdalny)
~/dane/fasta	katalog roboczy (~ to katalog domowy)
\$	koniec promptu; tu wpisujemy polecenie

Rzut oka na prompt mówi, **kim** jesteśmy, **gdzie** jesteśmy i na jakiej maszynie — warto wyrobić ten nawyk.

Polecenie ma stały wzorzec: **nazwa**, potem **opcje**, potem **argumenty**.

```
grep -c ">" probka_C.fasta
```

Część	Przykład	Rola
Polecenie	grep	co zrobić (tu: wyszukaj)
Opcja	-c	jak zmodyfikować działanie (tu: policz dopasowania)
Argument	">"	na czym operować (tu: wzorzec do znalezienia)
Argument	probka_C.fasta	wejście (tu: plik)

Pomoc do dowolnego polecenia: `man grep` albo `grep --help`.

Wiele opcji ma dwie równoważne postacie — krótką (jedna litera) i długą (czytelną nazwę).

Krótką	Długa	Działanie (grep)
-c	--count	policz dopasowania
-i	--ignore-case	ignoruj wielkość liter
-v	--invert-match	pokaż wiersze <i>niepasujące</i>
-n	--line-number	dopisz numery wierszy

- Krótkie opcje można **łączyć**: `grep -cn` to to samo co `grep -c -n`
- Długie są czytelniejsze w skryptach — od razu widać, co robią

Nie trzeba pamiętać wszystkich opcji — trzeba wiedzieć, **gdzie je sprawdzić**.

Sposób	Do czego
<code>man polecenie</code>	pełny podręcznik (wyjście klawiszem <code>q</code>)
<code>polecenie --help</code>	szybkie streszczenie opcji
<code>tldr polecenie</code>	praktyczne przykłady użycia
<code>explainshell.com</code>	rozkłada dowolne polecenie na części

Reguła praktyczna

Zanim skopiują Państwo polecenie z internetu, warto sprawdzić w `man` albo na `explainshell`, co dokładnie robią jego opcje. To odróżnia rozumienie od zgadywania.

Skrót	Działanie
Tab	uzupełnij nazwę pliku lub polecenia
↑ / ↓	przewijaj wcześniejsze polecenia
Ctrl+R	przeszukaj historię poleceń
Ctrl+C	przerwij działające polecenie
Ctrl+A / Ctrl+E	skocz na początek / koniec wiersza
Ctrl+L	wyczyść ekran

Klawisz Tab

Wpisujemy kilka pierwszych liter i wciskamy **Tab** — powłoka dokańcza nazwę. Chroni przed literówkami w długich nazwach plików.

Historia

Powłoka pamięta wpisane polecenia. Strzałką w górę wracamy do poprzedniego, a **Ctrl+R** wyszukujemy dawne polecenie po fragmencie.

4. Praca z plikami i katalogami

Pliki tworzą drzewo katalogów. Żeby wskazać plik, podajemy jego **ścieżkę**.

- **Katalog roboczy** — miejsce, w którym „stoimy”; sprawdzamy je poleceniem `pwd`
- **Ścieżka bezwzględna** zaczyna się od `/` — pełny adres od korzenia, np.
`/home/ala/dane/probka.fasta`
- **Ścieżka względna** liczona jest od katalogu roboczego, np. `fasta/probka.fasta`

Skróty warte zapamiętania

`~` to katalog domowy, `.` to katalog bieżący, `..` to katalog nadrzędny (o poziom wyżej).

Poruszanie się po katalogach

```
pwd          # pokaż katalog roboczy
ls -lah      # wypisz zawartość
cd fasta     # wejdź do katalogu fasta
cd ..        # cofnij się o poziom wyżej
```

- `pwd` — pokaż katalog roboczy (ang. *print working directory*)
- `ls` — wypisz zawartość: `-l` szczegóły, `-a` też pliki ukryte, `-h` czytelne rozmiary
- `cd` — wejdź do katalogu; `cd ..` cofa o poziom, samo `cd` wraca do katalogu domowego

Reguła praktyczna

Zanim uruchomią Państwo polecenie na plikach, warto sprawdzić `pwd` i `ls` — czy jesteśmy tam, gdzie myślimy, i czy pliki mają spodziewane nazwy.

Polecenie	Do czego
<code>cat plik</code>	wypisz całą zawartość
<code>less plik</code>	przeglądaj stronami (wyjście klawiszem <code>q</code>)
<code>head plik</code>	pierwsze 10 wierszy (<code>head -n 3</code> — trzy)
<code>tail plik</code>	ostatnie 10 wierszy (<code>tail -n +2</code> — od drugiego)
<code>wc -l plik</code>	policz wiersze

Wielkie pliki (gigabajty) otwieramy przez `less` albo `head`, nigdy przez `cat` — inaczej zalejemy ekran setkami tysięcy wierszy.

Polecenie	Działanie
<code>mkdir wyniki</code>	utwórz katalog
<code>touch plik.txt</code>	utwórz pusty plik (lub odśwież datę)
<code>cp zrodlo cel</code>	skopiuj plik
<code>cp -r kat1 kat2</code>	skopiuj cały katalog
<code>mv stara nowa</code>	przenieś lub zmień nazwę

`mv` służy zarówno do przenoszenia, jak i do zmiany nazwy — to ta sama operacja.

```
rm plik.txt
```

```
rm -r stary_katalog
```

- `rm` usuwa plik
- `rm -r` usuwa katalog z zawartością

Nie ma kosza

W terminalu `rm` kasuje **nieodwracalnie** — plik nie trafia do kosza. Szczególnie groźne jest `rm -rf` z błędną ścieżką.

Reguła praktyczna

Przed usunięciem warto wypisać to samo poleceniem `ls`, żeby zobaczyć, co dokładnie zniknie. Lepiej sprawdzić dwa razy, niż odtwarzać dane.

Znaki wieloznaczne (globbing)

Powłoka potrafi dopasować wiele plików naraz według wzorca — bez wypisywania każdego z osobna.

Wzorzec	Dopasowuje
<code>*.fasta</code>	wszystkie pliki z rozszerzeniem <code>.fasta</code>
<code>probka_?.txt</code>	<code>probka_A.txt</code> , <code>probka_B.txt</code> , ...(jeden znak)
<code>probka_[AB].txt</code>	tylko <code>probka_A.txt</code> i <code>probka_B.txt</code>
<code>dane/*</code>	cała zawartość katalogu <code>dane</code>

```
ls *.fasta          # wypisz wszystkie pliki FASTA
wc -l probka_*.txt  # policz wiersze w kilku plikach naraz
```

Powłoka rozdziela polecenie **spacjami**, więc spacja w nazwie pliku bywa źródłem błędów.

Problem

```
cat moja probka.txt
```

Powłoka widzi *dwa* argumenty: `moja` i `probka.txt`.

Rozwiązanie

```
cat "moja probka.txt"
```

Cudzysłów scala nazwę w jeden argument.

Reguła praktyczna

Najprościej **nie używać spacji** w nazwach plików roboczych — zamiast nich podkreślenie (`moja_probka.txt`). Oszczędza to kłopotów w całym kursie.

Każdy plik ma zapisane, kto może go czytać, zmieniać i uruchamiać. Pokazuje to `ls -l`.

```
-rwxr-xr-- 1 ala grupa 2048 analiza.sh
```

- Pierwsze znaki to prawa: **r** — odczyt, **w** — zapis, **x** — uruchamianie
- Żeby uruchomić własny skrypt, trzeba nadać mu prawo wykonywania:

```
chmod +x analiza.sh
```

Do uprawnień wrócimy przy skryptach (Moduły 3–4) — tu wystarczy wiedzieć, że istnieją i po co.

Szukanie plików: find

Gdy nie pamiętamy, gdzie leży plik, **find** przeszukuje drzewo katalogów według kryteriów.

```
find . -name "*.fastq"      # wszystkie pliki .fastq od bieżącego katalogu w dół
find dane -type d           # tylko katalogi wewnątrz dane/
```

- Pierwszy argument to miejsce startu (. — tu, gdzie jesteśmy)
- **-name** dopasowuje nazwę (wzorec w cudzysłowie), **-type f** pliki, **-type d** katalogi
- Szuka **rekurencyjnie**, więc zagląda też do podkatalogów

Reguła praktyczna

find świetnie łączy się z potokiem: listę znalezionych plików można przekazać dalej i przetworzyć je hurtem.

Ile miejsca zajmują dane: du i df

Dane sekwencyjne bywają wielkie — warto sprawdzać zajętość dysku, zanim skończy się miejsce.

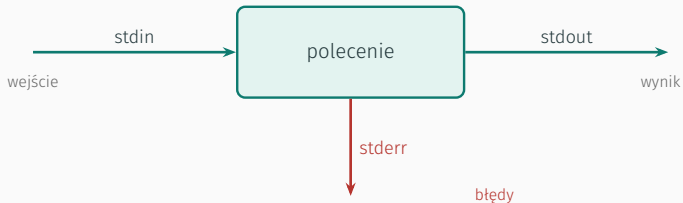
Polecenie	Do czego
<code>du -sh katalog</code>	rozmiar katalogu w czytelnej formie
<code>du -sh *</code>	rozmiar każdego elementu bieżącego katalogu
<code>df -h</code>	wolne miejsce na zamontowanych dyskach

Opcja `-h` (*human-readable*) pokazuje rozmiary jako `4,0K`, `2,3G` zamiast liczby bajtów — dużo czytelniej przy dużych plikach.

5. Filozofia Uniksa: potoki i filtry

Trzy strumienie każdego polecenia

Polecenie czyta z jednego wejścia i pisze do dwóch osobnych wyjść.



- **stdout** i **stderr** to dwa *różne* wyjścia — wynik osobno, komunikaty o błędach osobno
- Dzięki temu można zapisać wynik do pliku, a błędy nadal widzieć na ekranie

Zapis	Znaczenie
<code>polecenie > plik</code>	zapisz wynik do pliku (nadpisuje)
<code>polecenie >> plik</code>	dopisz wynik na koniec pliku
<code>polecenie < plik</code>	podaj plik jako wejście
<code>polecenie 2> błędy.log</code>	przekieruj komunikaty błędów

```
grep ">" probka_C.fasta > naglowki.txt
```

Uwaga

Pojedyncze > **nadpisuje** plik bez pytania. Gdy chcemy dopisać, używamy >>.

Filozofia Uniksa

Każde narzędzie robi **jedną rzecz dobrze**. Narzędzia porozumiewają się **tekstem**. Złożone zadania powstają przez **łączenie** małych narzędzi, nie przez budowę jednego wielkiego programu.

- Zamiast jednego programu „do wszystkiego” mamy kilkadziesiąt małych, wyspecjalizowanych
- Wspólnym interfejsem jest zwykły **tekst** — wyjście jednego narzędzia jest wejściem następnego
- Ten sam zestaw narzędzi działa na pliku z tabelą, z sekwencjami i z wariantami

Znak **potoku** (|) kieruje wynik jednego polecenia wprost na wejście następnego, bez pliku pośredniego.



Każdy człon dostaje strumień tekstu, przetwarza go i przekazuje dalej. Tak z prostych klocków budujemy złożone przetwarzanie.

Potok nie wczytuje całego pliku do pamięci ani nie tworzy plików pośrednich — dane **płyną** przez kolejne narzędzia.

- Każdy człon przetwarza strumień **kawałek po kawałku** i od razu podaje dalej
- Nie powstają wielkie pliki tymczasowe między krokami, więc nie zapycha się dysk
- Dzięki temu plik większy niż pamięć komputera i tak da się przetworzyć

Reguła praktyczna

To dlatego jeden potok potrafi policzyć odczyty w wielogigabajtowym FASTQ, choć Excel nie otworzyłby nawet ułamka takiego pliku.

Tabela `probki.tsv` ma kolumny `id`, `plik`, `grupa`. Policzmy liczbę próbek w każdej grupie:

```
cut -f3 probki.tsv | tail -n +2 | sort | uniq -c
```

Wynik:

```
2 kontrola
2 stres
```

- `cut -f3` — wytnij trzecią kolumnę → `tail -n +2` — pomiń nagłówek
- `sort` — uporządkuj → `uniq -c` — policz powtórzenia

Narzędzie	Do czego
<code>grep</code>	znajdź wiersze pasujące do wzorca
<code>cut</code>	wytnij wybrane kolumny
<code>sort</code>	posortuj wiersze (<code>-n</code> liczbowo, <code>-k</code> po kolumnie)
<code>uniq</code>	scal powtórzenia (<code>-c</code> zlicz)
<code>tr</code>	zamień lub usuń znaki
<code>wc</code>	policz wiersze, słowa, znaki

To są klocki, z których w Module 1 budujemy wieloetapowe potoki.

`grep` wyszukuje wiersze pasujące do wzorca. Wzorce mogą być dokładne albo opisane **wyrażeniem regularnym**.

```
grep "^>" probka_C.fasta
```

```
>geneC1  
>geneC2  
>geneC3  
>geneC4
```

- `^>` to wzorzec: znak `>` **na początku wiersza** (kotwica `^`)
- W FASTA nagłówki zaczynają się od `>`, więc to prosty sposób, by je wszystkie wypisać
- `grep -c "^>"` zamiast wypisywać — **policzy** sekwencje w pliku

Sortowanie po kolumnie

`sort` domyślnie porządkuje alfabetycznie, całymi wierszami. Opcje pozwalają sortować liczbowo i po wybranej kolumnie.

```
sort -t$'\t' -k3 -n dane.tsv    # po 3. kolumnie, liczbowo
```

- `-n` — sortuj liczbowo (inaczej `10` trafi przed `2`)
- `-k3` — sortuj po trzeciej kolumnie
- `-t$'\t'` — ustaw tabulator jako separator kolumn
- `-r` — odwróć kolejność (malejąco)

Reguła praktyczna

Sortowanie liczbowe (`-n`) to najczęstsza pomyłka: bez niego `sort` porównuje znak po znaku i `10` wypada przed `2`.

Przykład: najczęstsze chromosomy w VCF

Które chromosomy mają najwięcej wariantów? Z klocków, które już znamy:

```
grep -v "^#" warianty.vcf | cut -f1 | sort | uniq -c | sort -rn
```

- `grep -v "^#"` — odrzuć wiersze nagłówka VCF
- `cut -f1` — weź pierwszą kolumnę (chromosom) → `sort | uniq -c` — policz
- `sort -rn` — ustaw malejąco, żeby najliczniejsze były na górze

sed

Edytor strumieniowy. Podmienia, usuwa i drukuje wybrane wiersze — świetny do zamian tekstu w locie.

awk

Mały język do danych w kolumnach. Filtruje, liczy i formatuje wiersz po wierszu; rozpoznaje pola `$1`, `$2`, ...

Przykład: średnia długość z kolumny

```
awk '{ suma += $2 } END { print suma/NR }' dane.tsv
```

Do awk i sed wracamy w Module 2.

Łączenie poleceń i kody wyjścia

Każde polecenie po zakończeniu zwraca **kod wyjścia**: 0 oznacza sukces, wartość różną od zera — błąd. Powłoka pozwala uzależnić kolejne kroki od tego kodu.

Zapis	Znaczenie
A ; B	wykonaj A, potem B (niezależnie od wyniku A)
A && B	wykonaj B tylko jeśli A zakończyło się sukcesem
A B	wykonaj B tylko jeśli A zawiodło

Przykład

```
fastqc probka.fastq && echo "gotowe" — komunikat pojawi się dopiero, gdy analiza się powiedzie.
```

tee

Rozgałęzia strumień: zapisuje wynik do pliku *i* przekazuje go dalej potokiem. Przydatne, gdy chcemy zachować krok pośredni.

xargs

Buduje polecenia z listy na wejściu — pozwala uruchomić to samo narzędzie na wielu plikach naraz.

Na tym etapie wystarczy wiedzieć, że istnieją i po co — praktyka czeka w kolejnych modułach.

6. Dane i formaty w bioinformatyce

Ogromna większość formatów w bioinformatyce to zwykły **tekst**, a nie zamknięty format binarny.

- **Czytelność** — plik można obejrzeć **head**-em i od razu zrozumieć strukturę
- **Uniwersalność** — te same narzędzia (**grep**, **cut**, **awk**) działają na każdym z nich
- **Strumieniowość** — plik czyta się kawałek po kawałku, więc rozmiar nie jest przeszkodą

Dlatego warsztat z wiersza poleceń przenosi się wprost na dane biologiczne: nauczysz się przetwarzać tabelę, potrafisz Państwo przetworzyć też VCF czy GFF.

Najprostszy format sekwencji. Każdy rekord to wiersz nagłówka (zaczyna się od >) i sekwencja pod nim.

```
>geneC1
ATGCTAGCTAGCTTACGATCGATCGTAGCTAGC
>geneC2
ATGGGCTTAAGCTAGCATCGATCG
```

- Przechowuje DNA, RNA albo białka
- Nagłówek to identyfikator (i często opis) sekwencji
- Policzenie sekwencji to po prostu policzenie wierszy z >

Format surowych odczytów z sekwenatora. Każdy odczyt to **cztery** wiersze.

```
@odczyt_001
ATGCTAGCTAGCTTACGATCG
+
IIIIIIIIHHHHGGGGFFFE
```

- Wiersz 1: identyfikator (zaczyna się od @)
- Wiersz 2: sekwencja Wiersz 3: separator +
- Wiersz 4: **jakość** każdej zasady, zakodowana znakami (skala Phred)

Skoro odczyt zajmuje cztery wiersze, liczba odczytów to liczba wierszy podzielona przez cztery.

Przykład: liczba odczytów w FASTQ

Skoro jeden odczyt to cztery wiersze, liczbę odczytów policzymy jednym potokiem — także dla pliku spakowanego:

```
zcat probka.fastq.gz | wc -l          # liczba wierszy
```

- `zcat` wypisuje zawartość pliku `.gz` bez rozpakowywania go na dysk
- `wc -l` liczy wiersze; liczbę odczytów dostajemy, dzieląc wynik przez **cztery**
- Ten sam plik może mieć dziesiątki milionów odczytów, a potok i tak czyta go strumieniowo

Reguła praktyczna

To typowa *kontrola jakości na wejściu*: zanim ruszymy analizę, warto sprawdzić, ile odczytów faktycznie zawiera każda próbka.

Format	Zawartość
TSV/CSV	zwykłe tabele (kolumny rozdzielone tabulatorem/przecinkiem)
GFF/GTF	anotacja genomu: gdzie leżą geny, egzony, cechy
BED	interwały genomowe: chromosom, początek, koniec
VCF	warianty: pozycje różnic względem genomu odniesienia
SAM/BAM	odczyty dopasowane do genomu (BAM to binarna wersja SAM)

Formaty i narzędzia do nich (**samtools**, **bcftools**, **bedtools**) omawiamy w Module 5.

Format opisujący **różnice** próbki względem genomu odniesienia. Zaczyna się od wierszy nagłówka (**##**), potem następuje tabela.

#CHROM	POS	ID	REF	ALT	QUAL	FILTER
chr1	10583	.	G	A	52.4	PASS
chr1	12783	.	A	G	38.1	PASS

- Jeden wiersz to jeden wariant: gdzie (**CHROM**, **POS**) i jaka zmiana (**REF**→**ALT**)
- To wciąż tabela tekstowa, więc **grep** i **awk** działają tu tak samo jak na zwykłym TSV
- Policzenie wariantów na chromosomie to ten sam potok, co liczenie próbek w grupie

Format opisujący, **gdzie na genomie** leżą cechy: geny, egzony, sekwencje regulatorowe. Dziewięć kolumn rozdzielonych tabulatorem.

```
chr1  havana  gene    11869  14409  .  +  .  gene_id "ENSG..."
chr1  havana  exon    11869  12227  .  +  .  gene_id "ENSG..."
```

- Kluczowe kolumny: **chromosom**, **typ cechy**, **początek**, **koniec**, **nić** (+/-)
- Współrzędne liczone **od 1**, przedział obustronnie domknięty
- Zliczenie genów to policzenie wierszy, w których trzecia kolumna to **gene**

BED — interwały genomowe

Minimalistyczny format do opisu **fragmentów genomu**. W najprostszej postaci trzy kolumny.

chr1	1000	5000
chr1	8000	9500
chr2	200	1750

- Kolumny: **chromosom**, **początek**, **koniec** (opcjonalnie nazwa, wynik, nić)
- **Uwaga**: współrzędne liczone **od 0**, koniec wyłączony — inaczej niż w GFF
- Narzędzie **bedtools** liczy na tych interwałach części wspólne, pokrycia i sąsiedztwa

Typowa pułapka

BED liczy od zera, GFF od jedynki. Pomylenie konwencji przesuwają wynik o jedną pozycję — łatwo o cichy błąd.

Wynik **dopasowania odczytów** do genomu odniesienia. Każdy wiersz to jeden odczyt i miejsce, w które trafił.

- **SAM** to wersja tekstowa; **BAM** to jej skompresowana, binarna postać (mniejsza i szybsza)
- BAM jest binarny, więc nie oglądamy go wprost — do podglądu i filtrowania służy **samtools**
- Zawiera m.in. nazwę odczytu, chromosom, pozycję i jakość dopasowania

Reguła praktyczna

`samtools view plik.bam | head` zamienia binarny BAM na czytelny tekst i pokazuje pierwsze dopasowania — dalej działają już zwykłe filtry tekstowe.

Pliki skompresowane: .gz

Dane sekwencyjne bywają ogromne, więc trzyma się je spakowane — najczęściej z rozszerzeniem **.gz** (np. `probka.fastq.gz`).

Polecenie	Działanie
<code>gzip plik</code>	spakuj (tworzy <code>plik.gz</code>)
<code>gunzip plik.gz</code>	rozpakuj
<code>zcat plik.gz</code>	wypisz zawartość bez rozpakowywania
<code>zless, zgrep</code>	wersje <code>less/grep</code> dla plików <code>.gz</code>

Reguła praktyczna

Nie trzeba najpierw rozpakowywać pliku, żeby na niego zerknąć. `zcat probka.fastq.gz | head` pokaże początek bez tworzenia wielkiego pliku na dysku.

Reguła praktyczna

Skoro większość formatów to tekst w kolumnach, ten sam warsztat — potoki, **grep**, **cut**, **awk**, skrypty — obsługuje je wszystkie. Nauka narzędzi opłaca się raz i procentuje przy każdym formacie.

- Policzenie sekwencji w FASTA, wariantów per chromosom w VCF czy cech w GFF to warianty *tej samej* operacji
- Dlatego zaczynamy od fundamentu (M1–M4), zanim przejdziemy do wyspecjalizowanych narzędzi

7. Dobre praktyki i typowe pułapki

- Spacje w nazwach plików — powłoka rozdziela argumenty spacją; zamiast nich **podkreślenie**
- Ślepe `rm -rf` — brak kosza, kasuje nieodwracalnie; najpierw `ls`
- Pojedyncze `>` zamiast `>>` — nadpisuje plik bez ostrzeżenia
- Praca w złym katalogu — polecenie działa nie tam, gdzie myślimy; `pwd` rozwiewa wątpliwość
- Puste zmienne w ścieżkach — `rm -r $KAT/` przy pustym `$KAT` celuje w korzeń

Większość wypadków bierze się z pośpiechu, nie z braku wiedzy. Chwila sprawdzenia przed groźnym poleceniem to zawodowy nawyk.

- Sprawdzenie przed działaniem — `ls` i `pwd` przed uruchomieniem czegokolwiek na plikach
- Budowanie potoku stopniowo — człony dokładane po jednym, z kontrolą wyniku po każdym
- Uzupełnianie `Tab` i historia poleceń — mniej literówek, szybsza praca
- Proste nazewnictwo — bez spacji, bez polskich znaków, z rozszerzeniem mówiącym o formacie
- Zapisywanie działających poleceń — załączek przyszłego skryptu i dokumentacji analizy

Prosty, powtarzalny układ — uporządkowany projekt oszczędza godzin szukania:

```
projekt/  
├─ dane/          # surowe dane (tylko do odczytu)  
├─ wyniki/        # to, co wyprodukowały analizy  
├─ skrypty/        # polecenia i skrypty  
└─ README.txt     # co jest czym i jak uruchomić
```

- **Surowych danych nie ruszamy** — pracujemy na kopiach w **wyniki/**
- README to notatka dla przyszłego siebie: skąd dane, jak powstał wynik

Dobra nazwa mówi, co jest w środku, i dobrze współpracuje z powłoką.

Dobrze

```
probka_01_kontrola.fastq.gz  
liczba_wariantow_chr1.tsv  
2026-03-analiza_ekspresji.R
```

Źle

```
Próbka 1 (kopia).txt  
nowy plik FINAL v2.csv  
wyniki???.dat
```

Reguła praktyczna

Bez spacji i polskich znaków, konsekwentne rozszerzenia, daty w formacie RRRR-MM-DD (wtedy sortują się chronologicznie).

Orientacja i pliki

- `pwd`, `ls`, `cd` — gdzie jestem, co tu jest, wejdź
- `cat`, `less`, `head`, `tail` — podgląd
- `mkdir`, `cp`, `mv`, `rm` — porządki
- `find`, `du`, `df` — szukanie, miejsce

Potoki i filtry

- `grep` — szukaj wierszy
- `cut` — wytnij kolumny
- `sort`, `uniq -c` — porządkuj, zliczaj
- `wc -l` — policz wiersze
- `|`, `>`, `>>` — łącz i zapisuj

Pełną wersję (z opcjami i przykładami) znajdą Państwo w materiałach Modułu 1 na Moodle.

Podsumowanie

- Bioinformatyka to praca z danymi na dużą skalę — **wiersz poleceń** daje reprodukowalność, automatyzację i dostęp do serwerów
- Polecenie ma stały wzorzec: **nazwa, opcje, argumenty**; czyta ze **stdin**, pisze na **stdout** i **stderr**
- Filozofia Uniksa: **małe narzędzia** łączone **potokiem** (**|**) w złożone przetwarzanie
- Większość formatów biologicznych to **tekst**, więc jeden warsztat obsługuje je wszystkie

- **Praca rękami** — samo czytanie nie wystarczy; każde polecenie trzeba samodzielnie uruchomić
- **Podstawy bez pomijania** — Moduły 1–4 to fundament, na którym opiera się reszta
- **Błąd to informacja** — najpierw warto przeczytać komunikat; `--help` i `man` są pod ręką
- **Pytania na bieżąco** — na konsultacjach i forum kursu; lepiej dopytać, niż brnąć w ślepią uliczkę
- **Notatki z działających poleceń** — z czasem staną się podręcznym zestawieniem na całe studia

Narzędzia z tego kursu zostają na całą karierę: te same potoki obsługują pracę magisterską, doktorat i realne projekty badawcze.

Wykład 2: reprodukowalność i nowoczesny warsztat

Jak sprawić, by analiza działała nie tylko „u mnie”: skrypty i automatyzacja, kontrola wersji (Git), środowiska (Conda), kontenery (Docker), praca na klastrze, a także Python, Snakemake i R — czyli przegląd narzędzi całego kursu.

Przed kolejnymi modułami warto skonfigurować środowisko z Modułu 0 (terminal, Conda, edytor).

Dziękuję za uwagę

dr inż. Bartosz Czech · bartosz.czech@upwr.edu.pl